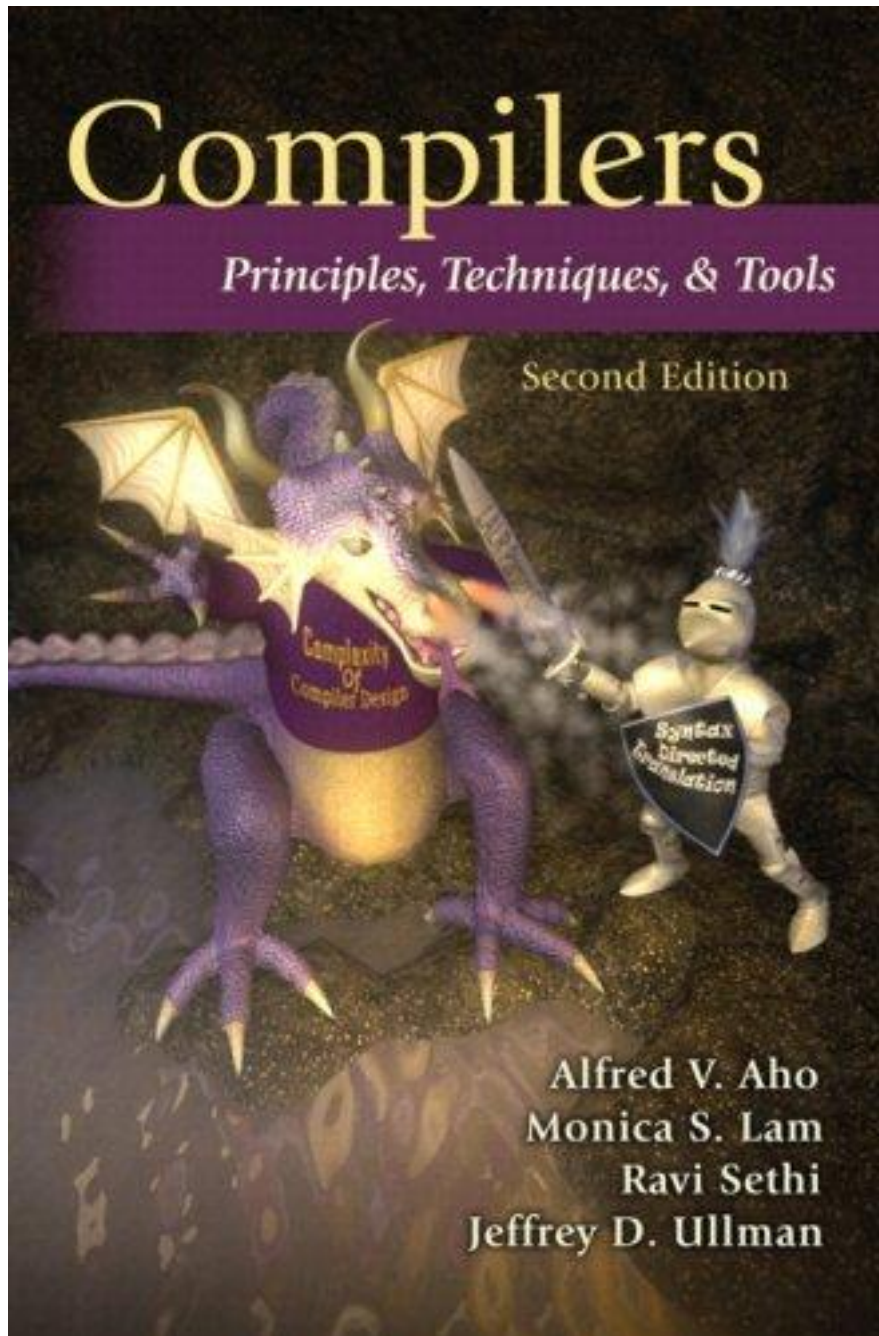




四川大學
SICHUAN UNIVERSITY

Principle of Compiler

luoyining@scu.edu.cn



Chapter 5

Syntax-Directed Translation

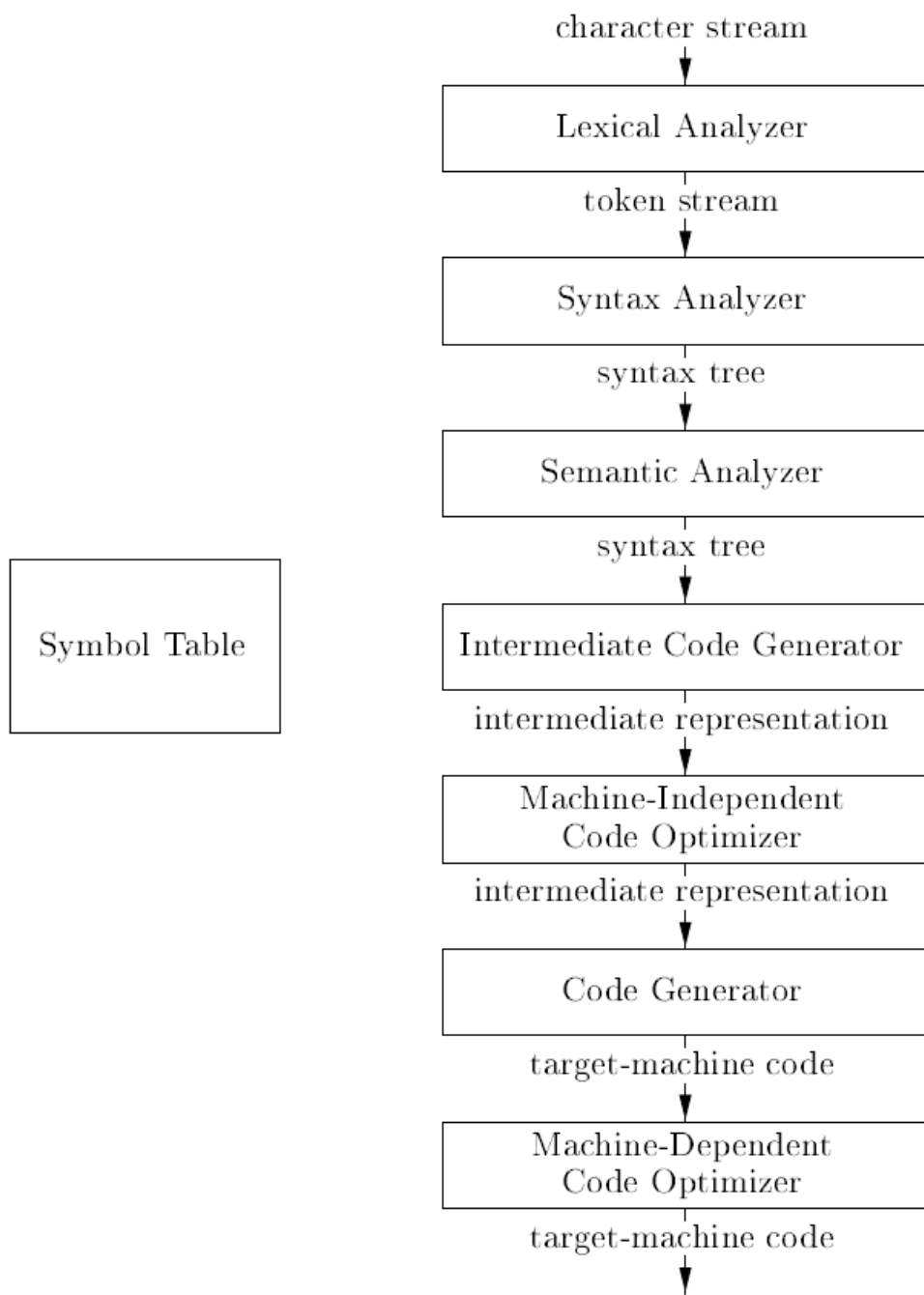


Figure 1.6: Phases of a compiler

3 Lexical Analysis

4 Syntax Analysis

5 Syntax-directed translation

- 语法制导翻译使用上下文无关文法来引导对语言的翻译。
 - 自然语言
 - 程序设计语言
 - 其他应用场景

Chapter 5

- SDD (Syntax-Directed Definition, 语法制导定义)
- SDT (Syntax-Directed Translation Schemes, 翻译方案)
- Applications
- Implementations

CONTENTS

5.1 Syntax-Directed Definitions

5.2 Evaluation Orders for SDD's

5.3 Applications of Syntax-Directed Translation

5.4 Syntax-Directed Translation Schemes

5.5 Implementing L-Attributed SDD's

语义的表示、计算和传递

- 什么是语义?

```
main()
{
    int u,v,temp;
    scanf("%d %d",&u,&v);
    if (v!=0)
    {
        do
        {
            temp=v; v=u-u/v*v; u=temp;
        } while (v!=0);
    }
    printf("u=%d\n",u);
}
```

$D \rightarrow TL$

$T \rightarrow \text{int} \mid \text{float}$

$L \rightarrow L, \text{id} \mid \text{id}$

$E \rightarrow E+T \mid E-T \mid T$

$T \rightarrow T * F \mid T / F \mid F$

$F \rightarrow (E) \mid \text{id} \mid \text{digit}$

Attributes and Rules

- 为文法符号设置**语义属性**，用来表示语法成分对应的语义信息。
- 在文法的产生式中附加**语义规则**，用来定义属性的计算和传递。
- 属性计算和传递的过程就是语义处理的过程。

$D \rightarrow TL$ $T \rightarrow \text{int} \mid \text{float}$ $L \rightarrow L, \text{id} \mid \text{id}$	• D • $T.type$ • $L.inh (L.type)$	<table><tr><th>PRODUCTION</th><th>SEMANTIC RULES</th></tr><tr><td>$D \rightarrow T L$</td><td>$L.inh = T.type$</td></tr><tr><td>$T \rightarrow \text{int}$</td><td>$T.type = \text{integer}$</td></tr><tr><td>$T \rightarrow \text{float}$</td><td>$T.type = \text{float}$</td></tr><tr><td>$L \rightarrow L_1, \text{id}$</td><td>$L_1.inh = L.inh$ $addType(\text{id.entry}, L.inh)$</td></tr><tr><td>$L \rightarrow \text{id}$</td><td>$addType(\text{id.entry}, L.inh)$</td></tr></table>	PRODUCTION	SEMANTIC RULES	$D \rightarrow T L$	$L.inh = T.type$	$T \rightarrow \text{int}$	$T.type = \text{integer}$	$T \rightarrow \text{float}$	$T.type = \text{float}$	$L \rightarrow L_1, \text{id}$	$L_1.inh = L.inh$ $addType(\text{id.entry}, L.inh)$	$L \rightarrow \text{id}$	$addType(\text{id.entry}, L.inh)$		
PRODUCTION	SEMANTIC RULES															
$D \rightarrow T L$	$L.inh = T.type$															
$T \rightarrow \text{int}$	$T.type = \text{integer}$															
$T \rightarrow \text{float}$	$T.type = \text{float}$															
$L \rightarrow L_1, \text{id}$	$L_1.inh = L.inh$ $addType(\text{id.entry}, L.inh)$															
$L \rightarrow \text{id}$	$addType(\text{id.entry}, L.inh)$															
$E \rightarrow E+T \mid T$ $T \rightarrow T * F \mid F$ $F \rightarrow (E) \mid \text{digit}$	• $E.val$ • $T.val$ • $F.val, \text{digit.lexval}$	<table><tr><th>PRODUCTION</th><th>SEMANTIC RULES</th></tr><tr><td>$E \rightarrow E_1 + T$</td><td>$E.val = E_1.val + T.val$</td></tr><tr><td>$E \rightarrow T$</td><td>$E.val = T.val$</td></tr><tr><td>$T \rightarrow T_1 * F$</td><td>$T.val = T_1.val \times F.val$</td></tr><tr><td>$T \rightarrow F$</td><td>$T.val = F.val$</td></tr><tr><td>$F \rightarrow (E)$</td><td>$F.val = E.val$</td></tr><tr><td>$F \rightarrow \text{digit}$</td><td>$F.val = \text{digit.lexval}$</td></tr></table>	PRODUCTION	SEMANTIC RULES	$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$	$E \rightarrow T$	$E.val = T.val$	$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$	$T \rightarrow F$	$T.val = F.val$	$F \rightarrow (E)$	$F.val = E.val$	$F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$
PRODUCTION	SEMANTIC RULES															
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$															
$E \rightarrow T$	$E.val = T.val$															
$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$															
$T \rightarrow F$	$T.val = F.val$															
$F \rightarrow (E)$	$F.val = E.val$															
$F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$															

Syntax Analysis & Semantic Analysis

- 显式地构造一棵语法分析树，然后通过遍历结点的方式来计算属性值；
- 翻译在语法分析的过程中交错完成，而不需要构造出显式的语法树。

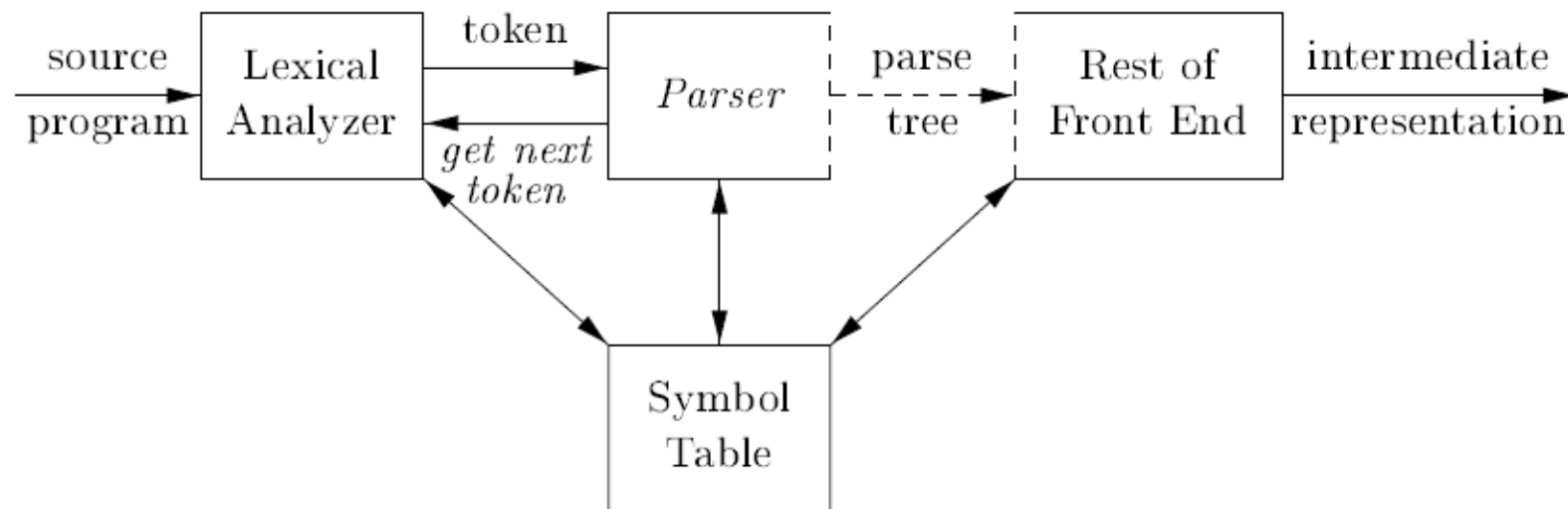


Figure 4.1: Position of parser in compiler model

5.1 Syntax-Directed Definitions

SDD

- 语法制导定义SDD是上下文无关文法和属性以及规则的结合。
- 属性和文法符号相关联，规则和生产式相关联。
- 属性和语义规则的设计

PRODUCTION	SEMANTIC RULES
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

PRODUCTION	SEMANTIC RULES
$D \rightarrow T L$	$L.inh = T.type$
$T \rightarrow \text{int}$	$T.type = \text{integer}$
$T \rightarrow \text{float}$	$T.type = \text{float}$
$L \rightarrow L_1 , \text{id}$	$L_1.inh = L.inh$ $addType(\text{id.entry}, L.inh)$
$L \rightarrow \text{id}$	$addType(\text{id.entry}, L.inh)$

Example 5.1

- 语义目的：简单计算器（计算表达式的值，并打印计算结果）
- 新增产生式 1) 是为了保证在根结点处打印出唯一的计算结果。
- 1. 属性设计
- 2. 语义规则设计

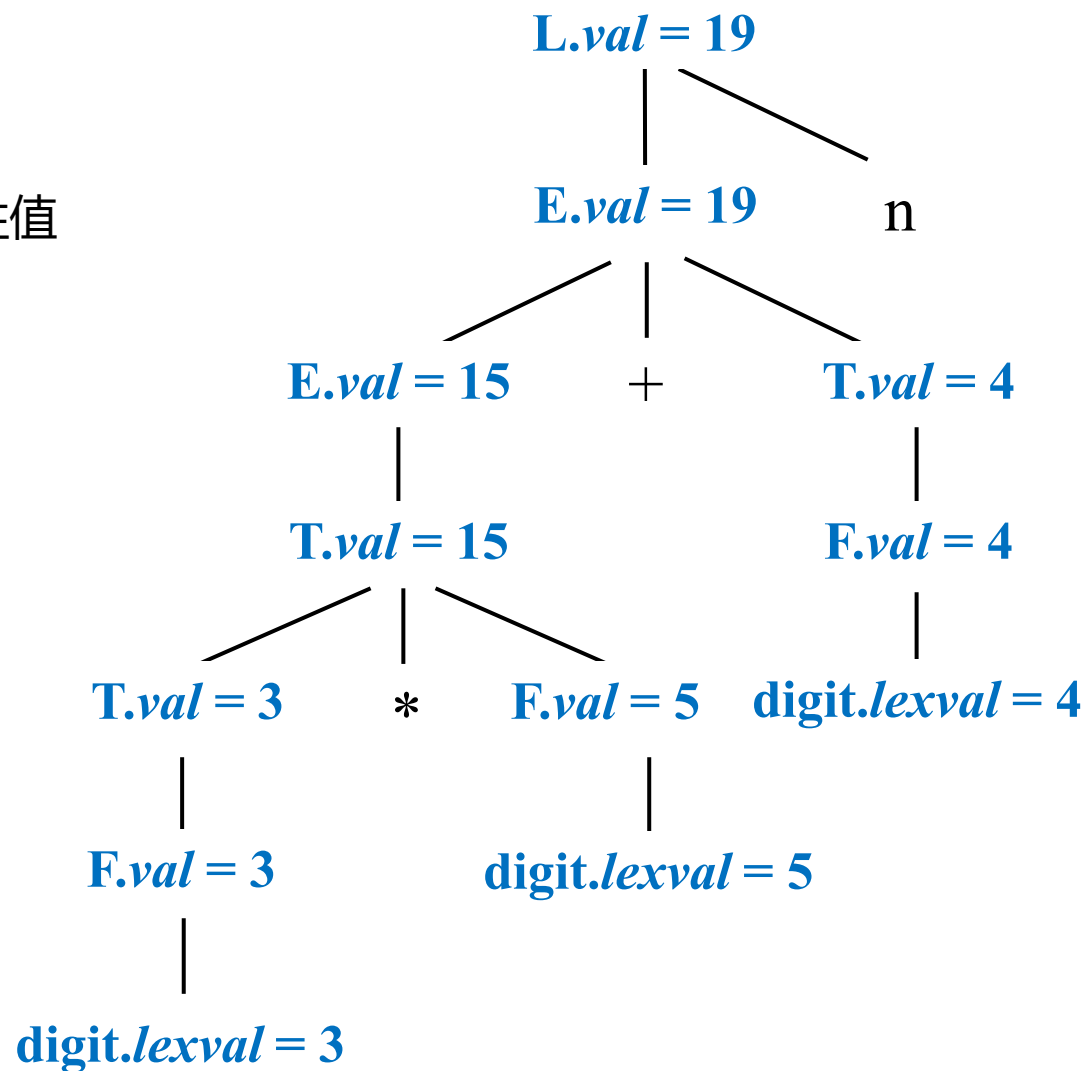
PRODUCTION	SEMANTIC RULES
1) $L \rightarrow E \text{ n}$	$L.val = E.val$ or <code>print($E.val$)</code>
2) $E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
3) $E \rightarrow T$	$E.val = T.val$
4) $T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
5) $T \rightarrow F$	$T.val = F.val$
6) $F \rightarrow (E)$	$F.val = E.val$
7) $F \rightarrow \text{digit}$	$F.val = \mathbf{digit.lexval}$

Figure 5.1: Syntax-directed definition of a simple desk calculator

Example 5.2

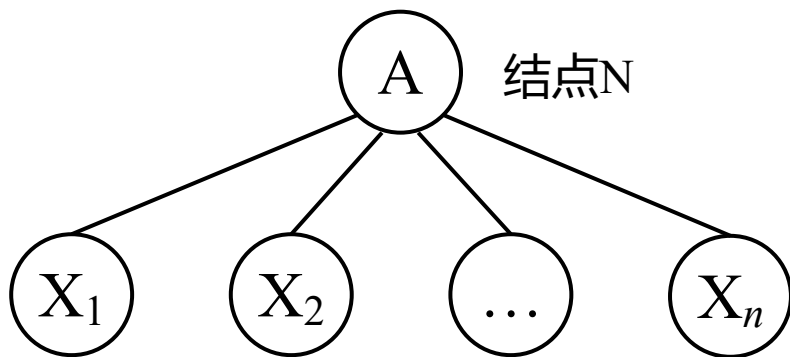
- 计算表达式 $3*5+4$ 的值
 - 在遍历语法树的过程中利用语义规则计算属性值
- 注释语法树(annotated parse tree)
- 自下而上传递信息

1) $L \rightarrow E \ n$	$L.val = E.val$
2) $E \rightarrow E_1 \ + \ T$	$E.val = E_1.val + T.val$
3) $E \rightarrow T$	$E.val = T.val$
4) $T \rightarrow T_1 \ * \ F$	$T.val = T_1.val \times F.val$
5) $T \rightarrow F$	$T.val = F.val$
6) $F \rightarrow (\ E \)$	$F.val = E.val$
7) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$



Synthesized Attributes

- 分析树结点N上的非终结符A的**综合属性**
 - 是由N上的产生式 $A \rightarrow X_1X_2...X_n$ 所关联的语义规则来定义的。
- **结点N上的综合属性值只能通过N的子结点以及N本身的属性值来确定。**

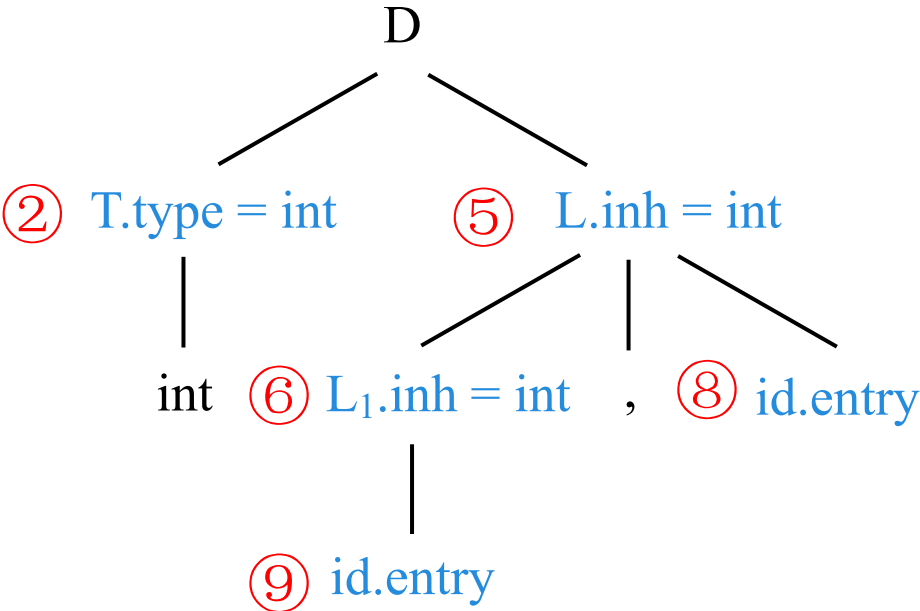


$$A.a = f(X_1.b, X_2.c, \dots, X_n.z, A.i)$$

Example 5.10

- 属性和语义规则的设计

- int a, b



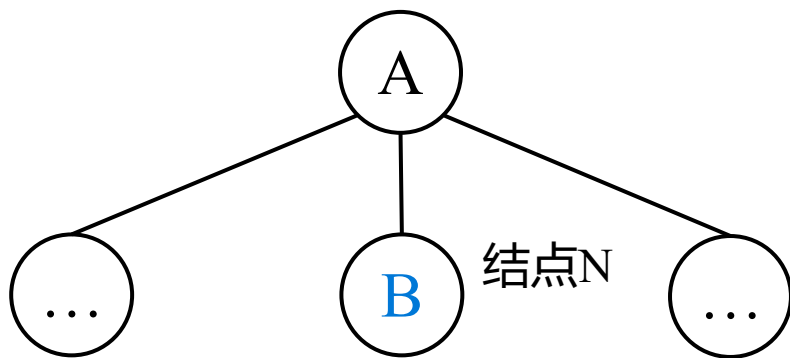
- $D \rightarrow T L$ ① **T.type**
- $T \rightarrow \text{int} \mid \text{float}$ ④ **L.inh** : 自上而上传递信息
- $L \rightarrow L_1, \text{id} \mid \text{id}$ ⑦ **id.entry** (id的符号表地址)

PRODUCTION	SEMANTIC RULES
1) $D \rightarrow T L$	$L.inh = T.type$ ⑤
2) $T \rightarrow \text{int}$	$T.type = \text{integer}$ ②
3) $T \rightarrow \text{float}$	$T.type = \text{float}$ ③
4) $L \rightarrow L_1, \text{id}$	$L_1.inh = L.inh$ ⑥ $addType(\text{id.entry}, L.inh)$ ⑧
5) $L \rightarrow \text{id}$	$addType(\text{id.entry}, L.inh)$ ⑨

Figure 5.8: Syntax-directed definition for simple type declarations

Inherited Attributes

- 结点N上的非终结符B的继承属性
 - 是由N的父结点上的产生式 $A \rightarrow X_1 \dots B \dots X_n$ 所关联的语义规则来定义的。
- 结点N上的继承属性值只能通过
 - N的父结点、N本身，以及N的兄弟结点的属性值来确定。



$$B.a = f(A.b, B.s, X_1.c, \dots, X_n.z)$$

5.1.1 Inherited and Synthesized Attributes

- 属性和文法符号相关联。一个符号可以有多个属性值。
 - V_N : 综合属性, 继承属性 综合属性是来自于子节点属性或自己的属性
 - V_T : 综合属性 继承属性来自于父节点与兄弟节点
- V_T 的综合属性值是由词法分析器提供的词法值, 在SDD中没有计算 V_T 属性值的语义规则。
- 属性有多种表现形式, 如数字、类型、符号表引用, 以及串等。
- 在一个SDD中, $\forall A \rightarrow \alpha \in P$, 都有一组相关联的语义规则, 每条规则的形式为 $b = f(c_1, c_2, \dots, c_k)$, 其中, f 是一个函数, b 和 $c_1, c_2, \dots, c_k$ 是该产生式文法符号的属性, 并且
 1. 如果 b 是 A 的一个属性, $c_1, c_2, \dots, c_k$ 是 α 中文法符号的属性或 A 的其他属性, 则称 b 是 A 的**综合属性**;
 2. 如果 b 是 α 中某个非终结符 X 的属性, $c_1, c_2, \dots, c_k$ 是 A 或 α 中文法符号的属性, 则称 b 是 X 的**继承属性**。在这两种情况下, 都称属性 b 依赖于 $c_1, c_2, \dots, c_k$ 。

Side Effects & Attribute Grammar

- 在实践中，翻译过程会出现一些副作用（side effects）。
 - 打印计算器的计算结果；
 - 和符号表进行交互。
- 允许使用受控的副作用。
- 属性文法（attribute grammar）：一个没有副作用的SDD有时也称为属性文法。
- Exercise 5.2.2:** For the SDD of Fig. 5.8, give annotated parse trees for the following string:

$L \rightarrow E n$	$L.val = E.val$ or <code>print(E.val)</code>
$L \rightarrow id$	<code>addtype(id.entry, L.inh)</code>

- 1) `int a, b, c`
- 2) `float w, x, y, z`

PRODUCTION	SEMANTIC RULES
1) $D \rightarrow T L$	$L.inh = T.type$
2) $T \rightarrow \text{int}$	$T.type = \text{integer}$
3) $T \rightarrow \text{float}$	$T.type = \text{float}$
4) $L \rightarrow L_1, id$	$L_1.inh = L.inh$ $addType(id.entry, L.inh)$
5) $L \rightarrow id$	$addType(id.entry, L.inh)$

Figure 5.8: Syntax-directed definition for simple type declarations

5.1.2 Evaluating an SDD at a Parse Tree

- 在语法树上对所有属性进行求值，有助于翻译过程的可视化（虽然实际上不需要构建语法树）。在计算过程中，属性值的计算是有依赖关系的。
- Example 5.2** Evaluate synthesized attributes in any bottom-up order.

1) $L \rightarrow E \mathbf{n}$	$L.val = E.val$
2) $E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
3) $E \rightarrow T$	$E.val = T.val$
4) $T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
5) $T \rightarrow F$	$T.val = F.val$
6) $F \rightarrow (E)$	$F.val = E.val$
7) $F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit.lexval}$

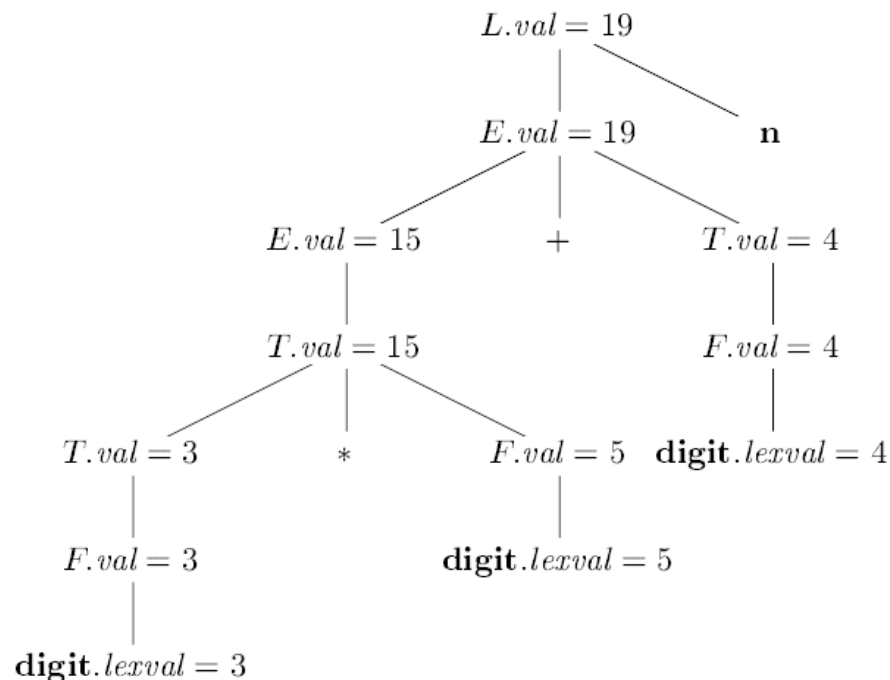


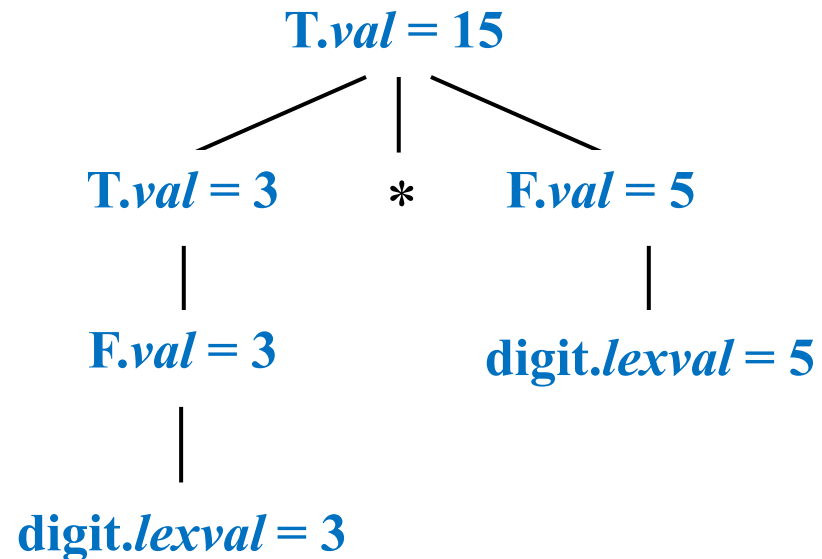
Figure 5.3: Annotated parse tree for $3 * 5 + 4 \mathbf{n}$

Example 5.3

- Left recursion grammar

$$T \rightarrow T * F \mid F$$
$$F \rightarrow \text{digit}$$

- 计算表达式 $3 * 5$ 的值



- Eliminate left recursion

$$T \rightarrow FT'$$
$$T' \rightarrow *FT' \mid \epsilon$$
$$F \rightarrow \text{digit}$$

Example 5.3

综合属性是来自于子节点属性或自己的属性
继承属性来自于父节点与兄弟节点

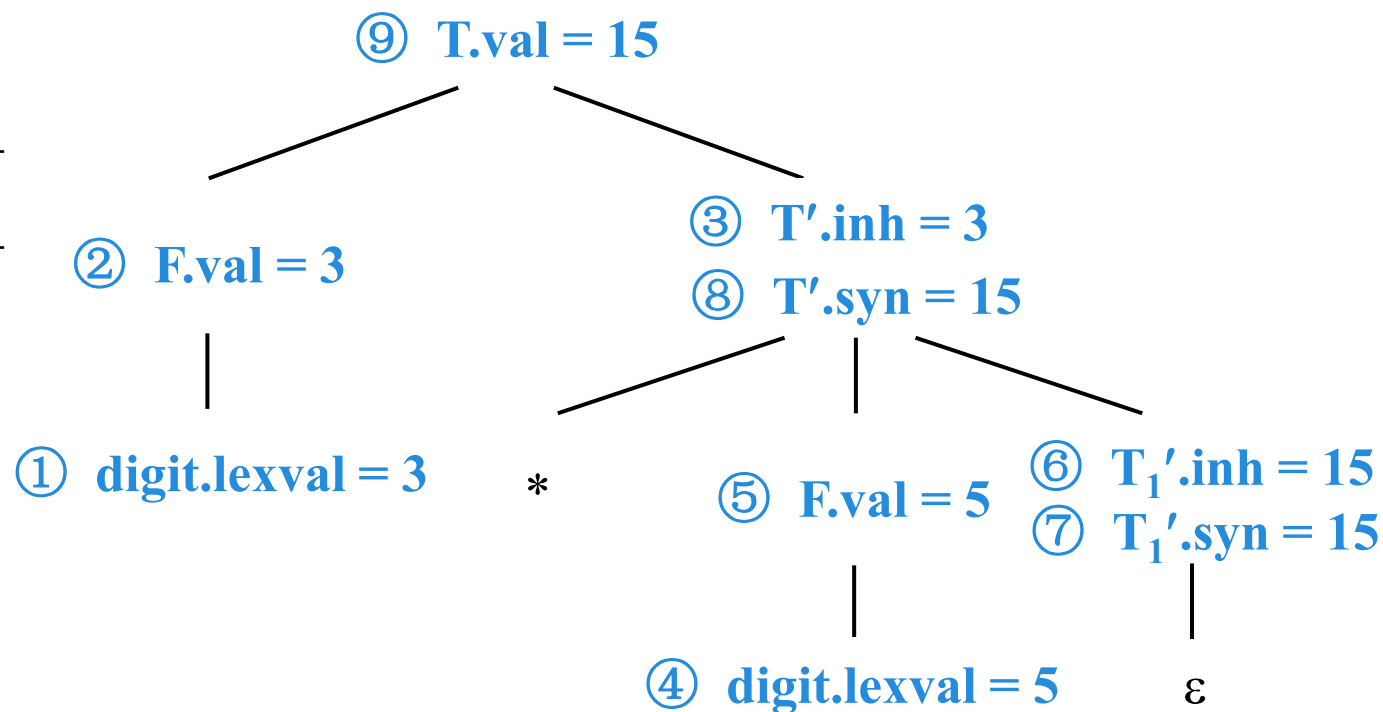
• 属性和语义规则的设计

$T \rightarrow FT'$ $T: \text{val}$

$T' \rightarrow *FT' \mid \varepsilon$ $T': \text{inh}, \text{syn}$

$F \rightarrow \text{digit}$ $F: \text{val}$

PRODUCTION	SEMANTIC RULES
1) $T \rightarrow F T'$	$T'.\text{inh} = F.\text{val}$ $T.\text{val} = T'.\text{syn}$
2) $T' \rightarrow * F T_1'$	$T_1'.\text{inh} = T'.\text{inh} \times F.\text{val}$ $T'.\text{syn} = T_1'.\text{syn}$
3) $T' \rightarrow \varepsilon$	$T'.\text{syn} = T'.\text{inh}$
4) $F \rightarrow \text{digit}$	$F.\text{val} = \text{digit}.\text{lexval}$



运算符 * 的左分量是通过继承得到的。

属性计算的依赖关系

- 语法制导定义没有规定一棵语法分析树中各个属性值的求值顺序。只要一个顺序能够保证在计算属性 a 的值时， a 所依赖的其他属性都已经计算完毕，这个顺序就是可以接受的。
- 一个SDD中，如果所有属性都是综合属性（Example 5.2），就可以按照任何自底向上的顺序进行求值计算。对同时具有继承属性和综合属性的SDD（Example 5.3），求值顺序比较复杂，并且不能保证一定有一个顺序对结点属性求值。
- 循环依赖关系

Production	Rules
$A \rightarrow B$	$A.s = B.i$ $B.i = A.s + 1$

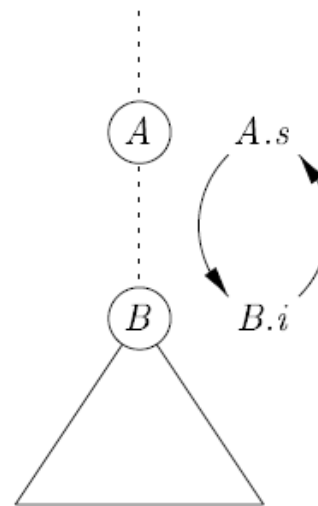


Figure 5.2: The circular dependency of $A.s$ and $B.i$ on one another

Exercises

- **Exercise 5.1.2:** Extend the SDD of Fig. 5.4 to handle expressions as in Fig. 5.1.

PRODUCTION	SEMANTIC RULES
1) $T \rightarrow F T'$	$T'.inh = F.val$ $T.val = T'.syn$
2) $T' \rightarrow * F T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3) $T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

Figure 5.4: An SDD based on a grammar suitable for top-down parsing

PRODUCTION	SEMANTIC RULES
1) $L \rightarrow E \mathbf{n}$	$L.val = E.val$
2) $E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
3) $E \rightarrow T$	$E.val = T.val$
4) $T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
5) $T \rightarrow F$	$T.val = F.val$
6) $F \rightarrow (E)$	$F.val = E.val$
7) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

Figure 5.1: Syntax-directed definition of a simple desk calculator

5.2 Evaluation Orders for SDD's

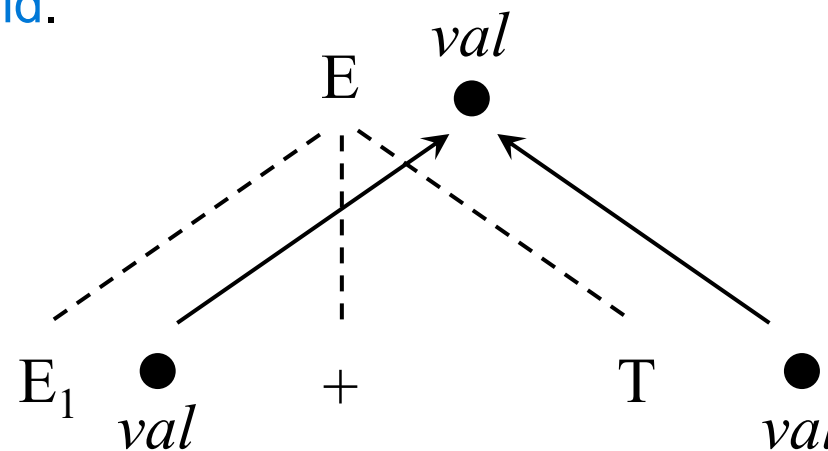
5.2.1 Dependency Graphs

- 依赖图是一个有向图，用来描述一棵语法树中结点的继承属性和综合属性之间的依赖关系。
- 通过依赖图，可以确定语法树中各个结点属性的求值顺序。

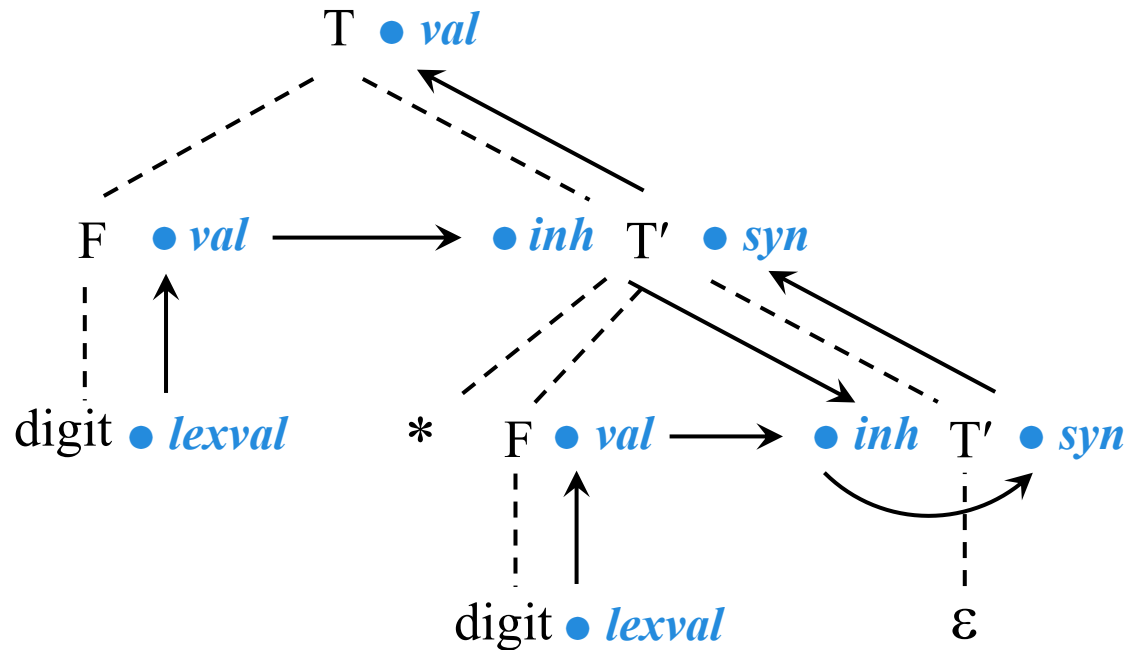
- **Example 5.4**

- The dependency graph has a node for each attribute associated with X.
- Show the parse tree edges as **dotted lines**, while the edges of the dependency graph are **solid**.

Production	Rules
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$



Example 5.5



Dependency graph for the annotated parse tree of Fig. 5.5

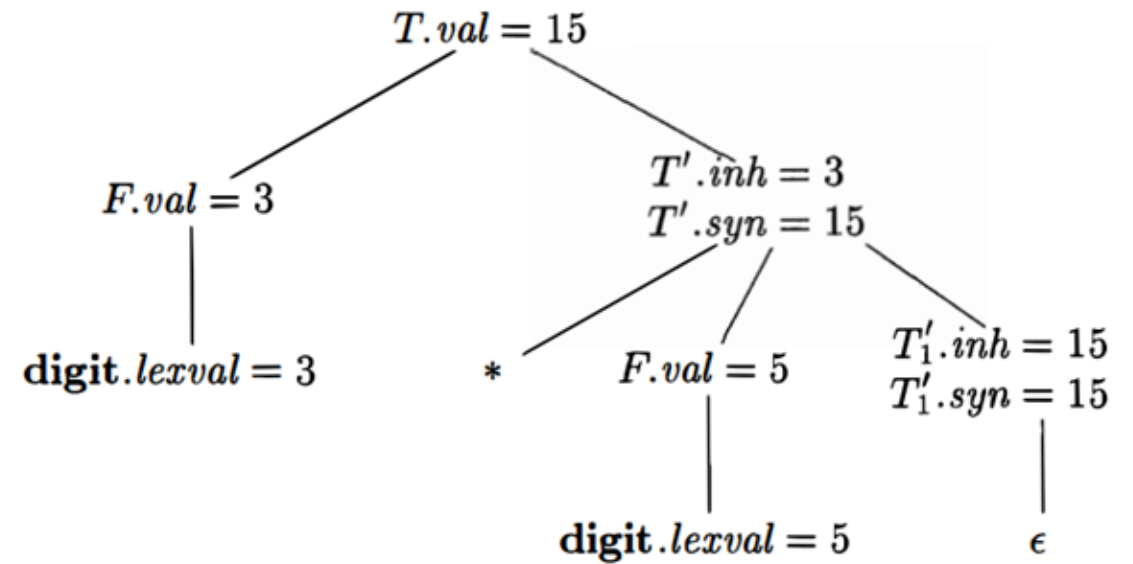
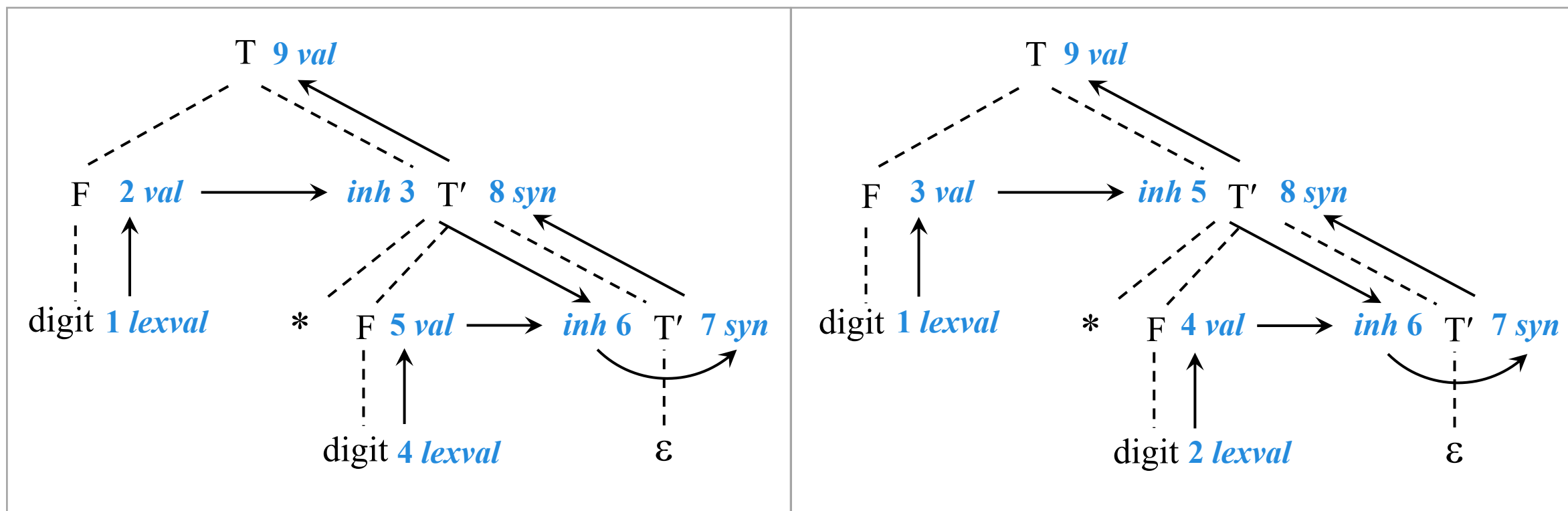


Figure 5.5: Annotated parse tree for $3 * 5$

Example 5.5

- 依赖图刻画了对语法分析树中结点的属性求值时可能采取的顺序。
- 求值顺序是否唯一？



Dependency graph for the annotated parse tree of Fig. 5.5

5.2.2 Ordering the Evaluation of Attributes

- 结点属性的一个可行求值顺序对应依赖图的一个拓扑排序 (topological sort) 。

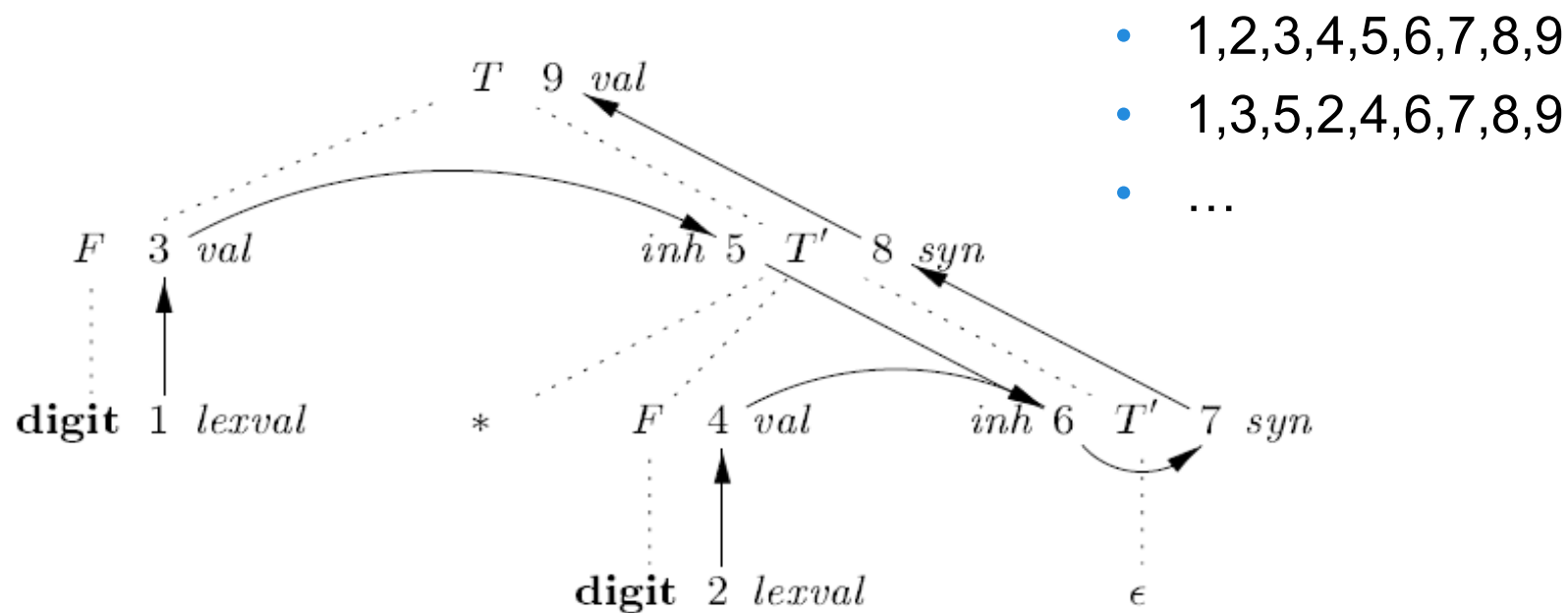
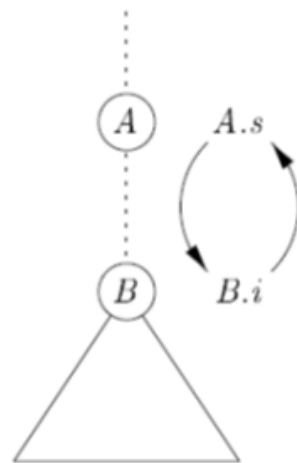


Figure 5.7: Dependency graph for the annotated parse tree of Fig. 5.5

Problem & Solution

- 如果图中有环，那就不存在拓扑排序。



- 从计算的角度来看，给定一个SDD，很难判定是否存在某棵语法树使得SDD的属性值之间具有循环依赖关系（其依赖图包含环）。
- 在实践中，翻译过程使用某些特定类型的SDD来实现。这类SDD不允许产生带环的依赖图，因此能够保证对每棵语法树都存在一个求值顺序。例如：S-SDD，L-SDD。

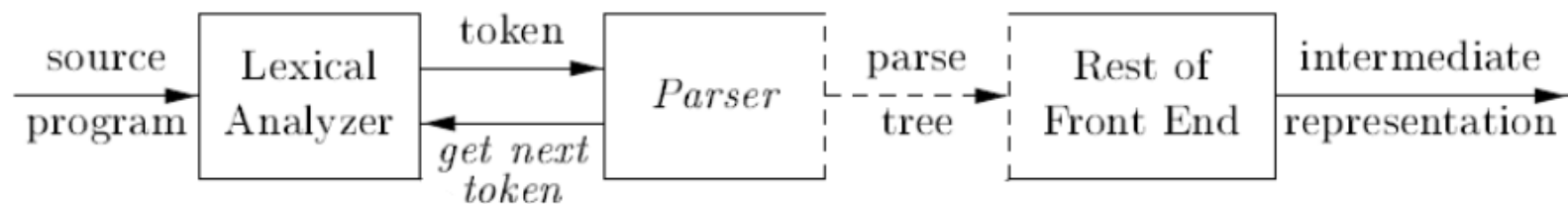
5.2.3 S-Attributed Definitions

- 如果一个SDD的每个属性都是综合属性，它就是S属性的（S-SDD）。
- **Example 5.7** $E.val$, $T.val$, $F.val$ 都是综合属性。

PRODUCTION	SEMANTIC RULES
1) $L \rightarrow E \mathbf{n}$	$L.val = E.val$
2) $E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
3) $E \rightarrow T$	$E.val = T.val$
4) $T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
5) $T \rightarrow F$	$T.val = F.val$
6) $F \rightarrow (E)$	$F.val = E.val$
7) $F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit.lexval}$

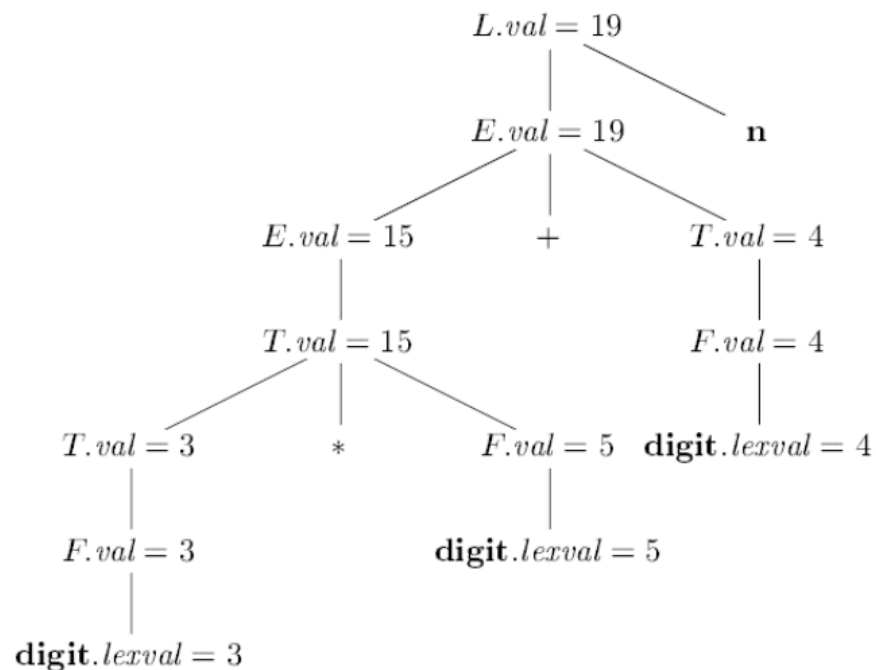
Figure 5.1: Syntax-directed definition of a simple desk calculator

S-SDD中属性的计算 - I



- 显式地构造一棵语法树
 - 可以按照任何自底向上的顺序在语法树中计算所有属性值；通常采用后序遍历。

```
postorder(N)
{
    for (each child C of N, from the left)
        postorder(C);
    evaluate the attributes associated with node N;
}
```



S-SDD中属性的计算 - II

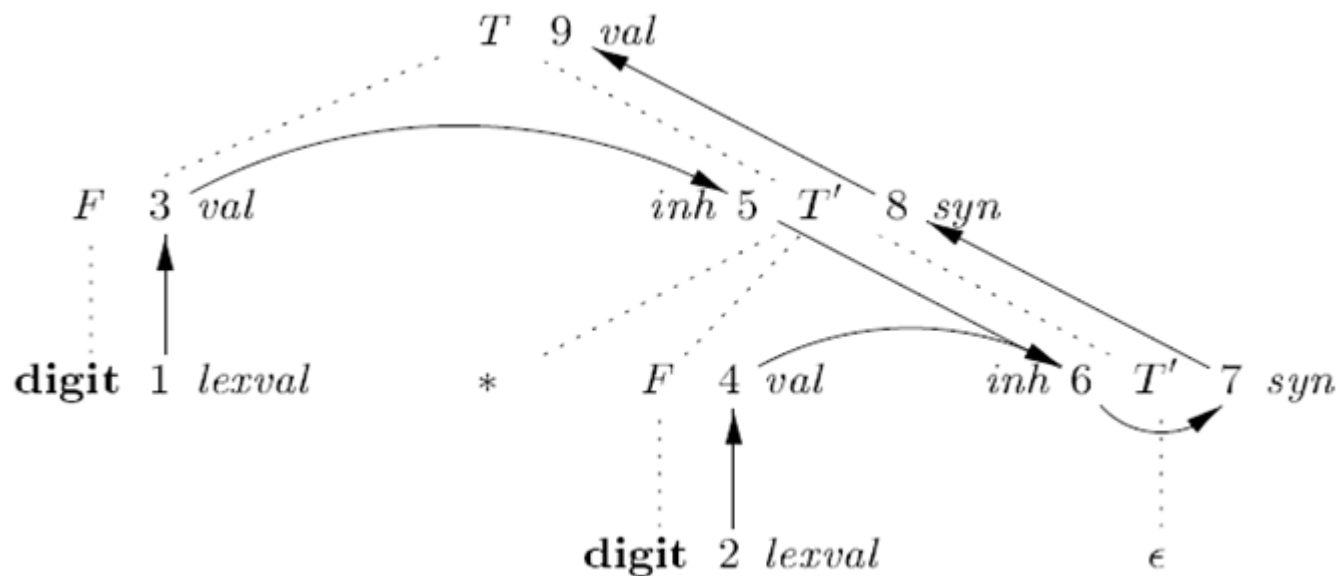
- 语法分析和语义分析交错进行，不构造显式的语法树
 - 可以在LR语法分析过程中计算S-SDD的所有属性值；
 - 后序顺序精准对应LR分析器归约的过程。

1) $L \rightarrow E \mathbf{n}$	$L.val = E.val$
2) $E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
3) $E \rightarrow T$	$E.val = T.val$
4) $T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
5) $T \rightarrow F$	$T.val = F.val$
6) $F \rightarrow (E)$	$F.val = E.val$
7) $F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit.lexval}$

	symbol	Input	Parsing Action	Semantic Action	Value
1	\$	3*5+4n	shift		
2	\$3	*5+4n	$F \rightarrow \mathbf{digit}$		
3	\$F	*5+4n	$T \rightarrow F$		
4	\$T	*5+4n	shift		
5	\$T*	5+4n	shift		
6	\$T*5	+4n	$F \rightarrow \mathbf{digit}$		
7	\$T*F	+4n	$T \rightarrow T*F$		
8	\$T	+4n	$E \rightarrow T$		
9	\$E	+4n	shift		
10	\$E+	4n	shift		
11	\$E+4	n	$F \rightarrow \mathbf{digit}$		
12	\$E+F	n	$T \rightarrow F$		
13	\$E+T	n	$E \rightarrow E+T$		
14	\$E	n	shift		
15	\$En		$L \rightarrow En$		
16	\$L				

5.2.4 L-Attributed Definitions

- Basic idea
 - Dependency-graph edges can go **from left to right**, but not from right to left (hence "L-attributed").
- SDD中如果出现继承属性，则在兄弟结点之间，只能依赖于左侧的兄弟结点。

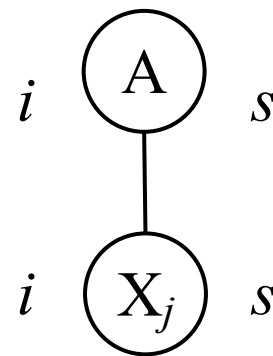


Definition

在一个产生式所关联的各属性之间，依赖图的边 (inh->syn) 可以从左到右，但不能从右到左。

- 在 L-SDD 中, $\forall A \rightarrow X_1X_2...X_n \in P$, 其语义规则中的每个属性
 - 或者是综合属性,
 - 或者是 X_j ($1 \leq j \leq n$) 的一个继承属性, 且这个继承属性仅依赖于:
 - (1) A 的继承属性;
 - (2) X_j 的左边符号 $X_1, X_2, \dots, X_{j-1}$ 的继承或综合属性;
 - (3) 自身的继承或综合属性, 但在该 X_j 的全部属性所组成的依赖图中不存在环。
- 为什么 L-SDD 中的继承属性不能依赖于父结点的综合属性?
- An S-SDD must be an L-SDD. ✓ or ✗

一个 S-SDD 一定是一个 L-SDD



Examples

- **Example 5.8** The following SDD is L-attributed. ✓ or ✗ 正确的

PRODUCTION	SEMANTIC RULES
1) $T \rightarrow F T'$	$T'.inh = F.val$ $T.val = T'.syn$
2) $T' \rightarrow * F T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3) $T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

- **Example 5.9** The SDD containing the following production and rules is L-attributed. ✓ or ✗

Production	Rules
$A \rightarrow B C$	$A.s = B.b$ $B.i = f(C.c, A.s)$

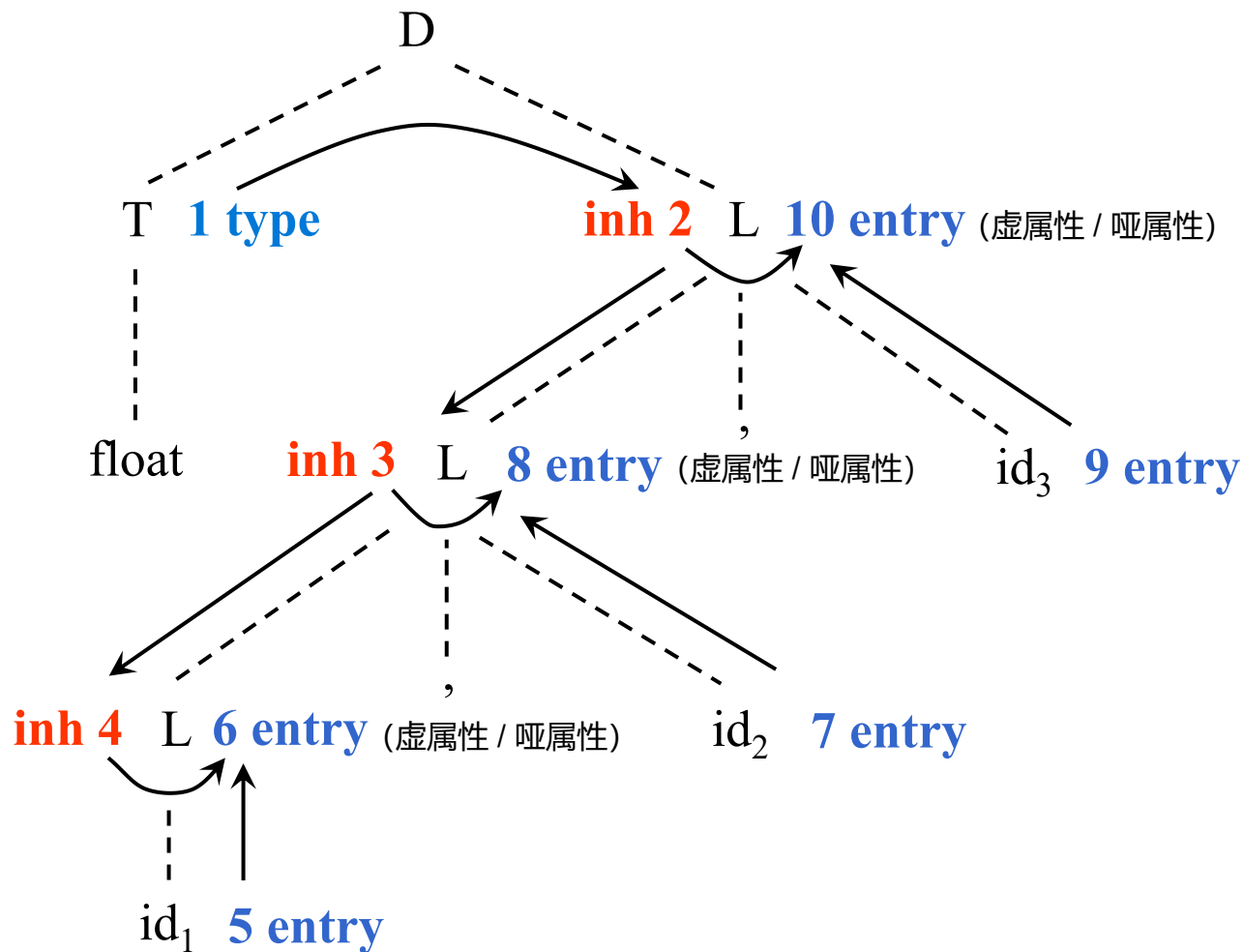
错误的，因为B.i从右边来自于A.s

Example 5.10

- Dependency graph for **float id1, id2, id3**

1) $D \rightarrow T \ L$	$L.inh = T.type$
2) $T \rightarrow \mathbf{int}$	$T.type = \text{integer}$
3) $T \rightarrow \mathbf{float}$	$T.type = \text{float}$
4) $L \rightarrow L_1, \mathbf{id}$	$L_1.inh = L.inh$ $addType(\mathbf{id}.entry, L.inh)$
5) $L \rightarrow \mathbf{id}$	$addType(\mathbf{id}.entry, L.inh)$

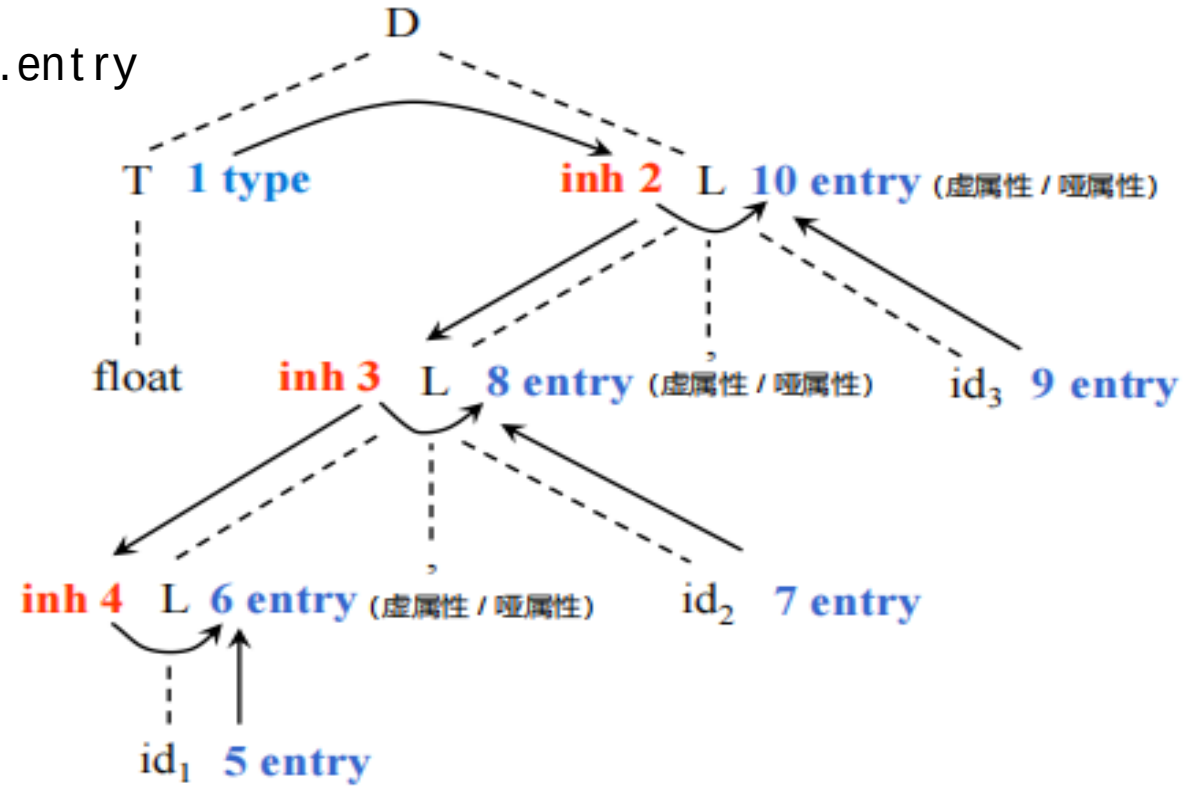
- L-SDD ? 是
- How to modify the SDD to check whether an identifier is declared more than once?



- How to modify the SDD to check whether an identifier is declared more than once?

初始化一个数组（或字典）类型的存储单元，在每一次id.entry时检查是否已经在数组（或字典）中存在

- 1) $D \rightarrow T L$
 $L.inh = T.type$
 $L.symbols = \emptyset$ // 初始为空集合
- 2) $T \rightarrow int$
 $T.type = integer$
- 3) $T \rightarrow float$
 $T.type = float$
- 4) $L \rightarrow L1, id$
 $L1.inh = L.inh$
 if id.entry in L1.symbols then
 error("identifier " + id.lexeme + " declared more than once")
 else
 addType(id.entry, L.inh)
 $L.symbols = L1.symbols \cup \{ id.entry \}$
- 5) $L \rightarrow id$
 addType(id.entry, L.inh)
 $L.symbols = \{ id.entry \}$



Example 5.10

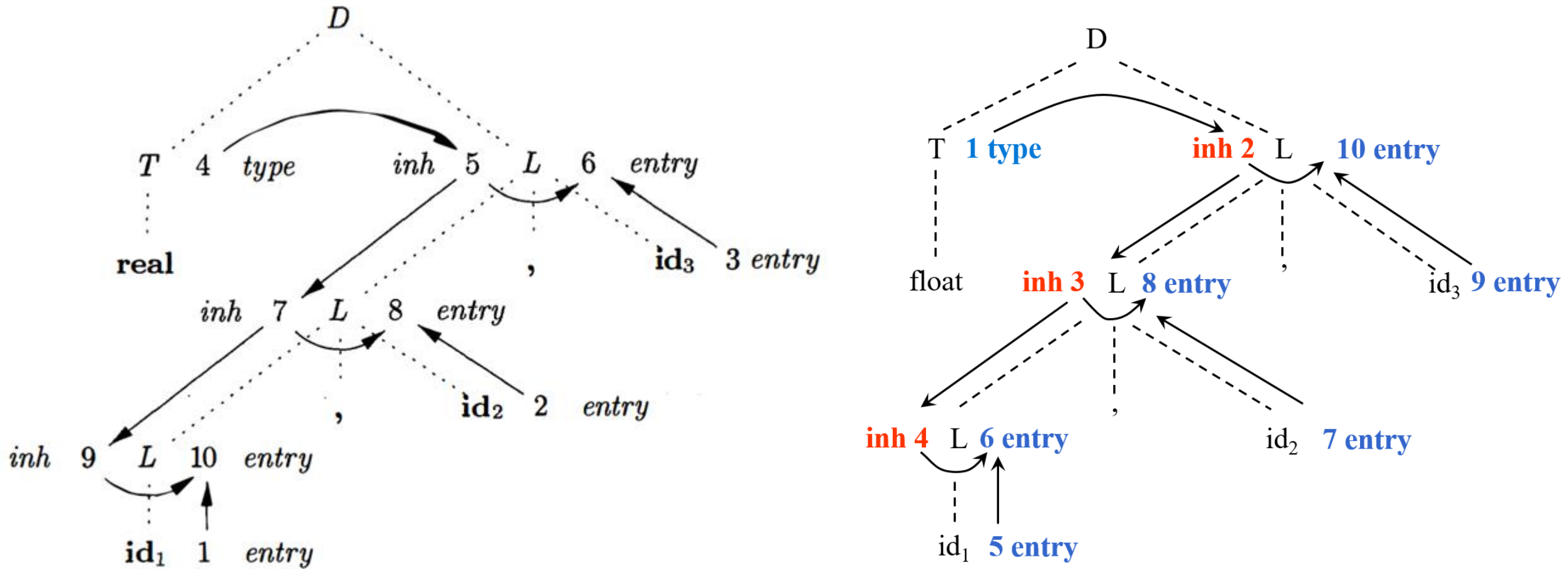


Figure 5.9: Dependency graph for a declaration `float id1, id2, id3`